

Experiment 4

Jenkins is an open-source automation server used for implementing Continuous Integration (CI) and Continuous Deployment (CD) pipelines. It helps automate the process of building, testing, and deploying applications, thereby improving development efficiency and software quality.

1. Overview of Jenkins

Jenkins allows developers to automatically integrate code changes into a shared repository and execute builds and tests. It supports a wide range of plugins that extend its functionality and enable integration with various tools and technologies.

Key features of Jenkins include:

- Continuous Integration and Continuous Deployment
- Plugin-based architecture
- Easy configuration through web interface
- Distributed build system
- Integration with version control systems like Git

2. Installation of Jenkins

Jenkins can be installed using different methods such as native installation or using Docker.

Using Docker

The command:

```
docker pull jenkins/jenkins:lts
```

is used to download the official Jenkins image.

To run Jenkins container:

```
docker run -p 8080:8080 -p 50000:50000 jenkins/jenkins:lts
```

Explanation:

- Downloads Jenkins Long Term Support (LTS) image
- Runs Jenkins in a container
- Port 8080 is used to access Jenkins web interface
- Port 50000 is used for agent communication

3. Initial Setup of Jenkins

After installation:

- Jenkins is accessed via browser using `http://localhost:8080`
- An initial admin password is required, which is stored inside the container
- Unlock Jenkins using the password
- Install suggested plugins
- Create an admin user

4. Jenkins Architecture

Jenkins follows a master-agent architecture:

- **Master (Controller):**
 - Manages jobs and scheduling
 - Provides web interface
 - Coordinates build execution
- **Agent (Node):**
 - Executes build tasks
 - Can be distributed across multiple systems

5. Integration with Tools

Jenkins integrates with multiple tools to automate workflows:

- **Version Control Systems:** Git, GitHub
- **Build Tools:** Maven, Gradle
- **Containerization:** Docker
- **Deployment Platforms:** Kubernetes, cloud services

Integration is achieved through plugins and pipeline scripts.

6. Jenkins Pipeline

A Jenkins pipeline is a set of automated steps defined in a script (Jenkinsfile) to execute CI/CD processes.

Types of pipelines:

- **Declarative Pipeline:** Structured and easy to read
- **Scripted Pipeline:** More flexible and uses Groovy scripting

Typical stages in a pipeline:

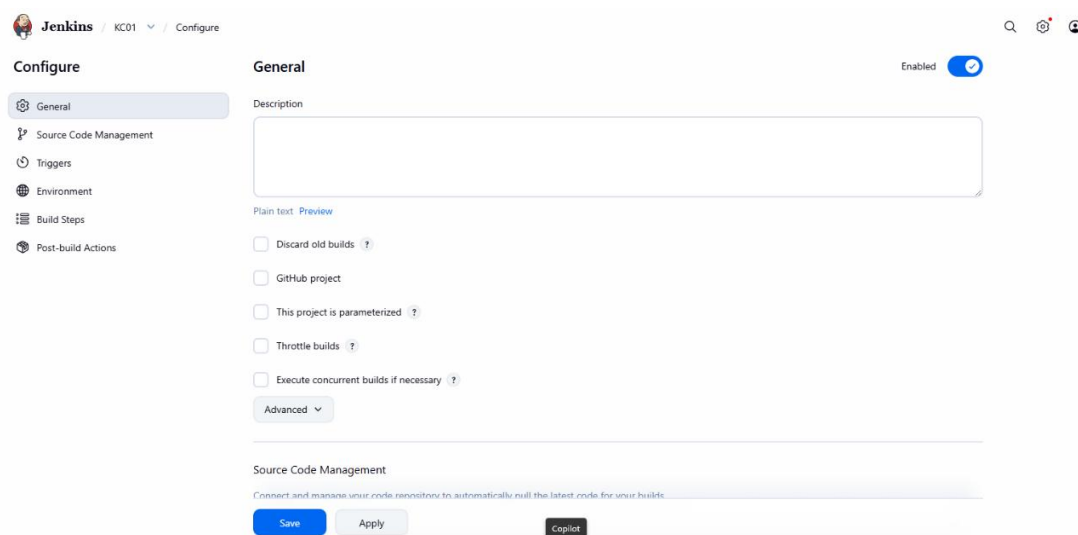
- Code checkout
- Build
- Test
- Deploy

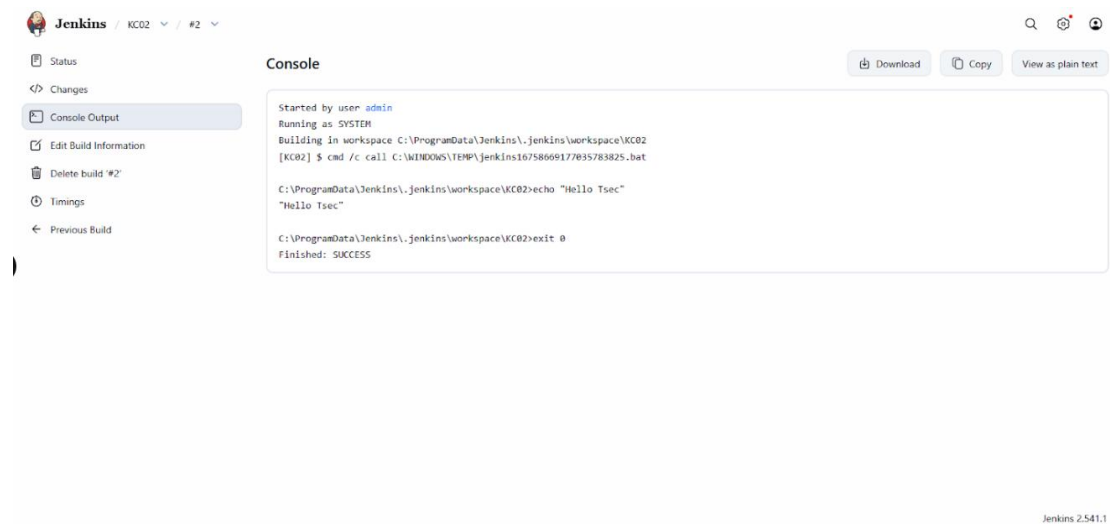
7. Benefits of Jenkins Integration

- Automates repetitive development tasks
- Detects errors early through continuous testing
- Improves collaboration among teams
- Speeds up software delivery process
- Ensures consistent deployment

Conclusion

Thus, Jenkins is a powerful tool for automating software development processes through Continuous Integration and Continuous Deployment. Its installation using Docker and integration with various tools enables efficient and reliable application development, testing, and deployment.





The screenshot shows the Jenkins web interface. At the top, the breadcrumb navigation indicates the job is 'Jenkins / KC02' and the current build is '#2'. On the left sidebar, the 'Console Output' tab is selected. The main console area displays the following text:

```
Started by user admin
Running as SYSTEM
Building in workspace C:\ProgramData\Jenkins\jenkins\workspace\KC02
[KC02] $ cmd /c call C:\WINDOWS\TEMP\jenkins16758669177635783825.bat

C:\ProgramData\Jenkins\jenkins\workspace\KC02>echo "Hello Tsec"
"Hello Tsec"

C:\ProgramData\Jenkins\jenkins\workspace\KC02>exit 0
Finished: SUCCESS
```

At the bottom right of the console area, the version 'Jenkins 2.541.1' is displayed.